在智能体编码框架上构建对话界面、智能记忆体与文献捕获体系

作者：** Zeng Pu · zengpu987@gmail.com · 独立研究者

摘要

智能体编码框架的界面以会话、终端与工具调用为中心，缺少三样东西：一个不调用工具的纯对话界面、一份跨会话持久的记忆、以及把检索到的文献收进工作流的入口。本文报告一套为 DeepSeek Harness 构建的扩展套件，它在不修改、不派生宿主源码的前提下补齐上述能力。套件由三个 bundle 组成：Chat 模式新增一个仿 ChatGPT 的对话面板，支持附件、Markdown 与 LaTeX 排版、模型与思考强度选择；Superman 记忆体把两个模式上传的文件、图片以及对话与任务记录归类进一个智能文件夹，并在每次组装提示词时把记忆摘要注入系统提示词；文献捕获在 arXiv 检索结果上增加「收藏到记忆体」按钮，下载失败时降级为一条记录。全部界面经宿主槽位注入，全部网络交 互走已鉴权的同源 `/api` 通道，不新开监听端口。浏览器半无法 import 宿主的 Client 包，因此 PDF 文本抽取、 LaTeX→MathML 渲染与编码识别都在本 bundle 内自行实现。评测给出：6 个文件共 7 187 行 JavaScript；5 套离线检 查共 284 项全部通过；14 个真实 PDF 抽出 199 942 个字符、67 235 个汉字、138 页，替换字符与 C1 控制字符为 0， 其中 9 个无文字层的文件被如实报告；编码探测 7/7 正确；16 条公式生成合法 MathML 且无反斜杠泄漏；三个侧边 栏入口在运行中的页面生效，并在运行中 Agent 自身的系统提示词里观察到记忆体摘要。

关键词：** 智能体编码框架；插件扩展；槽位系统；服务端发送事件；PDF 文本抽取；MathML；记忆体；词法检索

1 引言

智能体编码框架把模型放进一个带工具循环的会话里：用户描述任务，框架读写文件、执行命令、迭代修改代码。其界面因此是「会话 + 工具调用流」的形态，信息密度高，但对另一类需求并不友好——用户只想和模型讨论一段思路、解释一段代码、或把一个 PDF 丢进去提问，此时工具循环、权限确认与文件改动都没有必要。

我们以真实框架 DeepSeek Harness（下称 DSH）[1] 为宿主，用其插件机制补上三个缺口。**对话界面**：一个与 Agent 会话并列的、仿 ChatGPT 的纯对话面板，不携带任何工具，只做模型调用。**持久记忆**：智能体框架通常把上下文限制在单次会话里，跨会话复用信息要靠人手动粘贴；我们让两个模式上传的文件与产生的记录进入统一记忆体，两侧思考时都可被回忆。**文献捕获**：在设置里检索 arXiv 之后，用户应当能一键把论文收进资料库，而不是复制链接、下载、再手动上传。

这三件事共享同一组设计约束。DSH 的界面通过**槽位（slot）**组装，插件只能增加条目，不能替换随包 UI；客户端插件是以普通 ES 模块被模块加载器装载的，不能 import 宿主的 Client 包；插件与页面之间的通信必须走宿主已鉴权的 `/api` Fetch 通道，否则就要自行监听端口并处理凭据与来源校验。这些约束决定了实现形态：界面全部是新增条目，网络全部是同源路由，而所有在别处应当调用现成库的能力（PDF 解析、公式排版、编码探测）都必须自己写。

本文的贡献有四点：一套完全基于槽位、不替换随包 UI、不新开端口的扩展拓扑及其取舍；一个在浏览器内实现、不依赖任何 PDF 库的文本抽取流水线及其开发中真实暴露并修掉的五类缺陷；一个「索引 + 原文件」、写入串行且原子、同时服务于词法回忆与系统提示词注入的记忆体设计；以及对全部内容的离线验证方法学与实测数字，并如实说明它证明不了什么。

2 背景与相关工作

宿主与插件模型。 DSH 是一个以浏览器应用为界面的智能体编码框架，其界面由 React [14] 渲染，颜色与间距来自主题 token，文案走本地化服务。扩展点由 Cordis [2] 插件框架提供：一个 plugin 由 Host 半（Node 模块）与浏览器半（ES 模块）组成，通过 `apply(ctx, config)` 接入宿主上下文 `ctx`。我们使用的服务有 `ctx.slots`（界面槽位）、`ctx.connection.fetch`（已鉴权的 Fetch 路由注册）、`ctx.llm.stream`（模型流式调用）、`ctx.attachments`（附件存取）、`ctx.systemPrompt`（系统提示词分段）与 `ctx.on('session/event')`（会话事件订阅）。安装通过 profile 的 bundle 列表完成，宿主源码零改动。

PDF 文本抽取。 PDF 的对象模型、流过滤器与 CMap 机制由 ISO 32000-1 [3] 与 Adobe 的 PDF Reference [4] 规定。成熟的实现会构建完整的交叉引用表与对象图；本文的目标是「在一个插件 bundle 里、对常见中文文档抽出可用文本」，因此只扫描 `N M obj` 形式的对象定义，用 `endstream` 定位数据、用 `/ToUnicode` CMap 还原字形码，并跳过图像流。这一取舍在评测中直接体现为：有文字层的文档处理迅速，扫描件被明确报告为无文本而不是产出乱码。

字符编码。 浏览器内建的 `TextDecoder` 实现了 WHATWG 编码标准 [6] 规定的 GBK、Big5、Shift_JIS 与 EUC-KR 解码器，因此「按候选编码逐一严格解码再打分」这条路径无需自带码表；本文的编码识别只是在标准解码器之上加了一层选择与打分策略。

公式排版。 在浏览器里渲染 LaTeX 有两条路：引入 KaTeX [9] 这类完整实现，或把 LaTeX 的一个子集翻译成 MathML [5] 交给浏览器排版。前者保真度更高，但要把字库与样式一并打包进 bundle；后者依赖浏览器与系统数学字体。我们选择后者，理由与代价见第 4 节。TeX 的排版语义以 The TeXbook [8] 为参照，字体优先使用 STIX Two Math [13]。

词法检索。 记忆体的回忆采用词元重叠加时间衰减的打分，其家族可追溯到 BM25 一类的概率检索框架 [10]。我们刻意没有实现完整的 BM25：需要的是「在用户提问时挑出至多若干条最相关的记忆并渲染成可读摘要」，而不是排序质量可比的检索系统；中文按字与二元组切分，其偏差在第 8 节讨论。

框架内的记忆方案。 常见的做法是把记忆写在会话文件、项目内约定文件或向量库里：前者随会话生命周期消失，后者需要额外服务与网络。本文的记忆体是本地优先的——一个 JSON 索引加一个原文件目录，Host 侧直接读，两个模式通过同源路由访问，Harness 模式则由 Agent 自己决定是否去读原文件。

3 系统架构

套件由三个 bundle 组成，各自有 Host 半与浏览器半。Host 半是普通 Node 模块，由 Cordis 加载器载入；浏览器半是普通 ES 模块，由客户端的模块图提供。图 1 给出整体拓扑。

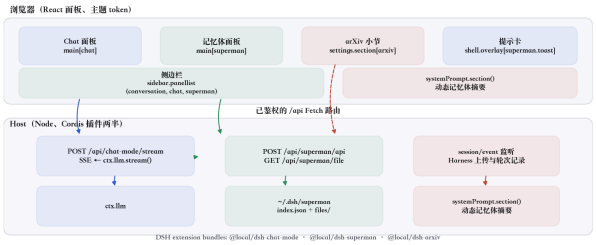


图 1. 扩展拓扑：浏览器面板、已鉴权的 Host 路由与 Cordis 插件两半。

槽位。 全部界面通过 `ctx.slots.inject(key, cb)` 与 `ctx.slots.register(options, component)` 注入。套件占用的槽位与条目见表 1。

表 1. 槽位占用：槽位、条目、次序与用途。

槽位	条目 id	顺序	用途
sidebar.panellist	conversation	-20	Harness 模式入口
sidebar.panellist	chat	-10	Chat 模式入口
sidebar.panellist	superman	-5	记忆体入口
main	chat	—	对话面板
main	superman	—	记忆体面板
settings.section	arxiv	20	arXiv 搜索小节
shell.overlay	superman.toast	—	收录提示卡

表 1 的关键性质是「只增不改」：`sidebar.panellist` 新增的三个条目 id 为 `conversation`、`chat`、`superman`，其中 `conversation` 不做界面工作——它的作用是让用户选中宿主自己的会话面板，因此无需复制随包 UI。次序上 `conversation` 为 −20、`chat` 为 −10、`superman` 为 −5，随包的 `plugins` 为 0，因此三个新模式排在插件与定时任务之前。`main` 槽新增 `chat` 与 `superman` 两个整页组件；`settings.section` 新增 id 为 `arxiv`、`order` 为 20 的设置节；`shell.overlay` 新增 id 为 `superman.toast` 的浮层，与随包的九个占用者并存。

传输。 两个 Host 半都通过 `ctx.connection.fetch.register` 在已鉴权的 `/api` 通道上注册**精确路径**的 Fetch 路由，而不是新开监听端口：

```
POST /api/chat-mode/stream        ■■■■■■■■■■SSE ■■
POST /api/superman/api            list/store/get/delete/note/recall ■■
GET  /api/superman/file?id=  ■■■■■■■
```

这一选择带来三项好处：继承浏览器的会话凭据与 Host/Origin 信任边界，无需自行实现鉴权；端口不与桌面应用、`dsh web` 冲突；路由随插件卸载一并撤销。代价是形态受限：只能声明精确路径，`requestBody` 需为 `'buffered'`。若自行监听端口，则要处理凭据、跨域与生命周期，并把「同源」这一最强前提丢掉。

流式应答。 Chat 的 Host 半把模型流序列化成 `text/event-stream`：每一帧写成 `data: <JSON>\n\n`，由 `ReadableStream` 的控制器推入，帧类型包括 `text`、`reasoning`、`usage`、`finish` 与 `error`。页面用 `fetch` 拿到响应体后以 `getReader()` 增量读取，按 `\n\n` 重组帧——这正是 HTML 标准中服务端发送事件 [7] 的分帧方式，只不过读端由页面自己实现，因此帧到达即可更新当前那条消息。取消语义是这条路径上最易出错的地方：路由把 `request.signal` 同时传给 `ctx.llm.stream` 与帧写入函数，页面中断请求时，

AbortSignal 既终止上游模型调用，也让后续入队停止；写入包裹在 try/catch 中，因为页面已离开时入队会抛错，而这时唯一正确的动作是关闭流。用户按下停止按钮对应**中止请求**而非「停止渲染」，不会留下仍在计费的上游流。

热重载。浏览器半是纯 ES 模块，客户端 HMR 监视器每 500 ms 轮询每个 bundle 的 mtime 与 size，变更后把新模块推给页面并重新注册插件。Host 半是普通 Node 模块：新增 bundle 时加载器立即激活它，但修改**已安装**的 Host 半需要重启进程。这一不对称决定了开发节奏——Host 侧的校验、路由与存储逻辑每次迭代后需要重启，客户端侧可以边跑边改。

4 Chat 模式

面板由会话列表、消息流与输入区构成：列表按更新时间排序，标题取首条提问（超过 48 字符截断）；输入区含回形针、模型选择、思考强度选择、记忆体开关与发送/停止，支持拖拽与粘贴，单条消息最多 6 个附件。面板不携带工具，一次请求就是一次普通模型调用。

请求契约与失败码。Host 路由只接受 `POST`，请求体形如：

```
{ provider, model, reasoningEffort?, messages }
```

校验失败返回带稳定机器码的 JSON，面板按自己的语言措辞出错原因，失败码有 `method`、`body`、`provider-missing`、`model-missing`、`messages-missing`、`messages-too-many`、`messages-empty`、`text-too-long`、`image-rejected`。限额：单图不超过 8 MiB，单个文本部分不超过 400 000 字符，一次对话不超过 200 条消息。图片以 base64 到达，Host 半校验媒体类型属于 PNG/JPEG/WebP/GIF、解码后非空且不超上限，再交给 `ctx.attachments.saveImages` 变成耐久附件引用，作为 `{type:'image', attachment}` 块进入请求；助手回合补上模型来源。模型流中的错误终止帧被渲染成错误行。

附件处理。网页（`.html/.htm/.xhtml`）去掉注释、`script`、`style`、`noscript`、`template`、`svg` 与全部标签，只内联可见文字。Mathematica 笔记本（`.nb`）被解析成输入与文本单元：单元格按平衡括号切片，`RowBox`、`SuperscriptBox` 一类盒式结构被拍平成它们显示的代码，最多 400 个单元；`.wl/.wls/.m` 等源码按文本内联，解析失败退回原文。文本类文件先做编码识别再内联，识别到的编码显示在附件卡片上；识别为二进制的文件（PNG、zip、docx 等）被明确跳过并说明，而不是把 `File.text()` 的乱码塞进对话。

编码识别。`File.text()` 假定 UTF-8，GBK 笔记本、Big5 源码与无 BOM 的 UTF-16 导出都会变成乱码。识别顺序：字节序标记（UTF-8 BOM、UTF-16LE、UTF-16BE）优先；其次严格 UTF-8，用 `fatal: true` 的解码器试解，成功且不含 NUL 时打分，非负则采纳；再次无标记 UTF-16，取样前 8192 字节，若 NUL 在偶/奇位置的分布偏斜（合计超过样本八分之一）则按该方向尝试，可信度不低于 0.4 才采纳；最后在 `gbk`、`big5`、`shift_jis`、`euc-kr`、`windows-1252` 中逐一严格解码并取最优，最优分低于 0.1 即判为二进制。可信度是逐字符累加后除以字符数：替换字符与 C0/C1 控制字符各 −6，汉字与兼容表意文字 +3，假名与谚文 +2，可打印 ASCII 与通用标点 +1，拉丁扩展与修饰字母 −1——它把「哪种解码读起来更像文本」变成可比较的数，而不是靠启发式顺序碰运气。

PDF 文本抽取。抽取在本 bundle 内实现，不依赖任何 PDF 库，流水线分五级。

1. **对象扫描**。文件按「每字节一字符」处理，用 `(\d+)\s+(\d+)\s+obj` 找对象定义，再按其后的 `endobj`、`stream` 与字典闭合符确定字典文本与流数据范围。`/Length` 可能是间接引用（`12 0 R`），这一级无法解引用，因此先找 `endstream`，只有声明的长度既非负、不越界、也不越过 `endstream` 时才采信。图像流跳过。
2. **流解码**。按 `/Filter` 声明的逆序应用过滤器：`FlateDecode` 走平台的 `DecompressionStream`，`ASCII85Decode` 与 `ASCIIHexDecode` 自实现。PDF 压缩流常带尾部填充，平台解码器会把这截尾随垃圾视为错误并拒绝整段数据，因此 `FlateDecode` 改为手工逐块读取、把异常捕获在读取循环内部，异常前产出的字节恰是完整结果；字典未声明过滤器而首字节为 `0x78` 时也按 zlib 试解一次。单条解码流超过 8 MiB 即丢弃。
3. **字形映射**。非图像流若含 `beginbfchar` 或 `beginbfrange` 即按 `/ToUnicode` CMap 解析：前者为逐码映射，后者支持「递增基准」与「显式列表」，目标串按 UTF-16BE 十六进制串解析；多张 CMap 合并成一张表，供没有页树信息的文件兜底。
4. **内容流重放**。解析一个很小的文本算符子集：`Tf` 切换字体，`Tj`、`'`、`"` 输出字面串，`TJ` 输出数组（不大于 −180 的位移当作词间空格），`Td`、`TD`、`T*`、`Tm`、`BT`、`ET` 决定换行。串读成字节后交给当前字体的 CMap：双字节字体按两字节一码，单字节按字节取表。若字体声明 `WinAnsiEncoding` 却携带 GBK/Big5/Shift-JIS 字节，在「高字节占比超过 0.6，且按该编码解出的汉字占比高于 0.5、也高于原解码」时改用中文解码；这条兜底让许多国产排版工具产出的文件能被正确读出。抽取文本按行清理：合并空格、丢弃偏噪的行（替换字符、C1 控制字符或控制字符占比超过 0.4）、折叠多余空行。
5. **页面装配**。按 `/Type /Page` 找页对象，沿 `/Parent` 最多上溯 8 跳解析**继承的** `/Resources`，取出 `/Font` 中资源名到字体对象的映射，再按 `/Contents` 顺序重放内容流并按页拼接；若一个页对象都没有（页字典藏在对象流里，或结构不完整），则回退到「重放每一条含 `BT` 与 `Tj`/`TJ` 的流，使用合并后的 CMap」。质量门槛：文本不足 2 个字符即报 `no-text`，可读字符比例低于 0.75 即报 `unreadable`，只有跨过门槛的文本才会被内联。

这条流水线暴露并修掉了五类真实缺陷，它们都是靠语料上的失败现象定位的：

- **填充字节**：最初把流直接交给平台解码器，部分中文文档整页丢失文本；改为手工读取、在读取循环内吞掉尾随错误。
- **间接 `/Length: 12 0 R`** 被误读为数字 12，流数据被截断；改为无法解引用时以 `endstream` 定位。
- **`/Filter` 取值越界**：按正则提取过滤器名时越过本键的值边界，把后续键的一部分当成过滤器名，整条流被判为不支持的过滤器而丢弃；修法是只读一个完整的值（嵌套字典、数组、字面串、名称、引用或数字）。

- **继承的** `/Resources`：页对象自身没有 `/Resources`，字体映射在父节点上，以 `Tf` 切换字体后整页文本为空；改为沿 `/Parent` 上溯。
- `/ObjStm` **对象流**：页与字体字典被压缩进对象流，按对象定义扫描看不到它们；改为解析对象流的 `N`、`First` 头部与偏移表并并入对象表。

LaTeX→MathML。 修复前公式被当成纯文本原样显示，这正是「公式乱码」的来源。现在把 LaTeX 的一个子集翻译成 MathML [5]，由浏览器用系统数学字体 STIX Two Math [13] 排版，回退链为 `STIX Two Math` → `Latin Modern Math` → `Cambria Math` → `STIXGeneral`。翻译器是递归下降解析器，输出节点带 `xmlns` 与块级/行内 `display` 属性，支持分组与上下标（`msub`、`msup`、`msubsup`）、大算符上下限（`munder`、`mover`、`munderover`）、`\frac`、`\sqrt`、矩阵与 `cases`、`\left…\right` 伸缩括号、重音与向量、`\mathbb`/`\mathcal` 等字母表命令。分隔符识别 `$…$`、`$$…$$`、`\(…\)`、`\[…\]`，以及**没有分隔符的裸 LaTeX** 行——整行以 `\frac{…}` 或 `\begin{…}…\end{…}` 开头时按公式处理；行内 `$…$` 另有一层判断，纯数字、逗号、点、空格与百分号组成的体不进入公式路径，因此 `$100`、`$200` 这类金额不会被误判。解析失败降级为原文，不会白屏。我们没有内置 KaTeX [9]：完整的排版引擎、字库与样式会显著增加包体积。表 5 与图 6 给出 16 条公式电池的测量结果。

表 5. LaTeX→MathML 公式电池：16 条表达式的合法性、反斜杠泄漏与结构统计。

表达式	MathML 字符	分式	上下标	表格	合格
E = mc^2	161	0	1	0	☐
frac{partial f}{partial x	153	1	0	0	☐
vec{F} = mvec{a}	229	0	0	0	☐
sum_{n=1}^{infty} frac{1}	368	2	3	0	☐
int_{-infty}^{infty} e^{-	353	0	3	0	☐
oint_{partial Omega} vec	402	1	4	0	☐
begin{aligned} a &= b + c	281	0	0	1	☐
left[begin{matrix} 1 & 0	315	0	0	1	☐
… 共 16 条					16/16

Markdown 与对话记录。 回答按块渲染：代码围栏（含语言标签）、标题、列表、引用、表格、行内代码、链接、粗体、斜体与删除线；未识别的部分保持原文，因此半流式回答仍可读。会话历史存放在**本页面 origin 的** `localStorage`，键为 `dsh.chat-mode.history.v1`，是带版本号的 JSON 文档，含会话列表与每条会话的消息、模型选择。限额：最多 40 个会话、每会话 200 条消息、整列表 3 500 000 字符、每个文本附件保留 40 000 字符正文、推理轨迹 20 000 字符，图片只保留 160 px 缩略图。写入走 600 ms 去抖；超出配额时按最旧优先丢弃会话并重试，最多 6 轮，同时在面板上提示。选择本地存储是因为不必改 Host 代码、装好即用；代价是清除站点数据会清空记录，且它与另一个端口上的 `dsh web` 各存一份。

与记忆体的联动。 上传时同时把字节与已提取正文存入记忆体；每次发送前用这一轮的问题（截断到 600 字符）调一次 `recall`，命中时把摘要插在该条用户消息最前面；每完成一轮就把该会话镜像成一条 `kind: 'conversation'` 记录，id 为 `chat:<■■id>`，正文取最近 8000 字符。记忆体开关可关掉回忆，状态记在浏览器里；任意一半缺席时联动**静默降级**，对话不受影响。

5 Superman 记忆体

记忆体是侧边栏里的一个「智能文件夹」，图标为大脑，名称显示为 Superman，给两个模式提供同一份长期记忆：Harness 与 Chat 中上传过的文件与图片都进入其中，自动归类到 PDF / 图片 / 程序 / 其他四个文件夹；对话与任务记录同步进记忆体核心；两侧思考时都能回忆到其中内容，不同对话之间因此可以相互联想。

存储结构。 记忆目录由 `config.root` 决定，patch 里用 `dshHomePath('superman')`，默认 `~/.dsh/superman`：

```
/index.json      ■■■{ version, items[], notes[] }
/files/      ■■■■■■■id ■ sha256 ■■■■ 16 ■■■■■
```

所有索引写入串在同一个 Promise 链上，每次先写 `<index>.tmp` 再 `rename` 覆盖正式文件，因此两个并发上传不会写坏索引，进程在写入中途终止也不会留下半个 JSON。索引带版本号，版本不匹配或解析失败时从空索引重建，文件仍在 `files/` 下。

`Item` 记录 id、名称、分类、媒体类型、字节数、收录时间、来源（`chat` / `harness` / `superman` / `arxiv`）、来源标识、`detail`（人类可读的大小或像素尺寸）、是否有正文预览、预览长度，以及可选的宽高与页数；`Note` 记录 id、`kind`（`task` / `conversation`）、标题、正文、来源、会话 id、创建与更新时间、标签。正文预览每个条目最多 4000 字符，记录正文最多 8000 字符，记录数上限 500，单文件上限 64 MiB。

去重按内容哈希加文件名：相同 sha256 且同名的新上传不会新建条目，只刷新收录时间，因此同一次上传被两个模式各捕获一次时不会在记忆里留下两份。

分类级联。 归类的判断顺序是：

1. **魔术字节**：`%PDF-` → PDF；PNG、JPEG（`FF D8 FF`）、GIF、BMP、RIFF-WEBP → 图片；`PK\x03\x04` → 其他（压缩包）；
2. MIME：`application/pdf` → PDF；`image/*` → 图片；
3. **扩展名**：`pdf` → PDF；图片扩展名 → 图片；代码扩展名 → 程序（含 Mathematica 的 `nb`、`wl`、`m`，以及 `py`、`js`、`ts` 等一份较长的清单）；
4. **内容启发式**：前 4096 字节中控制字符占比超过 0.1 即判为其他（二进制）；否则按文本解码，以 `#!` 开头记 10 分，命中注释、括号分号、`def`/`function`/`class`/`import` 关键字、`=>`/`::`/`->`、HTML 标签、`#include`、`:=` 等特征各记 1 分，达到 2 分即判为程序；
5. 其余 → 其他。

图片尺寸**直接从文件头解析**，不引入任何图像库：PNG 读偏移 16 与 20 的两个大端 32 位整数；GIF 读偏移 6 与 8 的两个小端 16 位整数；JPEG 遍历段标记，遇到 `SOF0` – `SOF15` 中除 `C4`、`C8`、`CC` 之外的段即取高宽；WebP 在 `VP8X` 块上读 24 与 27 偏移处的两个 24 位小端整数并各加 1。解析失败只是没有尺寸，不影响其他

字段；卡片上因此可以显示 `1920×1080 · 2.4 MB` 这样的说明。

回忆打分。 查询先被词元化：拉丁词按非字母数字下划线加号点号井号切分，保留长度 2 – 40 的词；中文（含假名、谚文）逐个取字，并额外加入相邻两字的二元组。打分对条目是：名称命中每个词元 +4；正文（名称 + 分类 + 预览拼接）命中 +1.5（词元长度 ⩾2）或 +0.6；同一词元在正文中出现超过 3 次再 +0.5；最后加一个新鲜度项，等于 `min(1.5, max(0, 1 – ■■■■■ / 30 ■))`——即 30 天内的条目按线性衰减最多加 1.5 分，更旧的只保留词元分。记录的打分为标题命中 +3、正文命中 +1.2 或 +0.5。结果按分数降序，`limit` 默认 6，实现上钳制在 1 – 20。命中的片段是围绕第一个命中词元的 200 字符窗口（向前多取 60 字符并在截断处加省略号），空白被压平。

回忆结果被渲染成可直接塞进提示词的摘要：

```
■■■■ Superman■■■ N ■■
- [PDF] ■■.pdf■…■■■■■■■■■■…
■■■■■■■■■■■■■■■■■■ files/ ■■■■■■■■
```

提示词摘要与注入。 Host 半注册一个名为 `superman-memory` 的系统提示词分段，其 `text` 是一个**函数**（而不是字符串），因此每次组装提示词时都会重新计算摘要，永远反映记忆体的当前内容。该分段排在宿主 `FILE_REFERENCE` 分段之后（取其次序加一，取不到时为 901）。摘要内容是：记忆目录路径与「原文件在 files/、索引为 index.json」的说明、四个分类的条目数与记录数、最多 24 条最近收录条目（每条带 80 字符的正文片段并标注来源）、最多 6 条最近的对话/任务记录，以及一句明确指示：需要旧材料时直接读 `files/` 下的对应文件或 `index.json`，引用时说明依据记忆体的哪一项。评测时我们在这套扩展自身所在的 Agent 会话的系统提示词里观察到了这段摘要（详见第 7 节）。

双模式捕获。 Chat 侧的上传由面板主动 `POST`，Harness 侧则由 Host 半的 `session/event` 监听器捕获。监听器处理三类事件：`user/message` 时记录该会话的首问（截断到 300 字符）并扫描消息内容里的附件引用；`assistant/message` 时更新最近回答（截断到 400 字符）并同样扫描；`turn/end` 时把轮次加一，写一条 id 为 `session:<■■id>` 的任务记录，正文由「轮次、首问、最近回答」三行组成，并以会话工作目录作为标签。附件扫描是一个深度不超过 6 的递归遍历，凡同时带有 `attachmentId` 与 `mediaType` 或 `name` 的对象都被视为附件引用；已处理过的附件 id 记在一个集合里，因此同一条会话日志不会把同一附件重复收录。图片经 `attachments.readImage` 读出，其他文件经 `attachments.readFileStream` 读出后合并写入，来源标记为 `harness`。整个监听器包在 try/catch 中：记忆体出错必须被吞掉，绝不能因为记忆写入失败而让会话追加失败。

收录提示。 记忆体守望者每 4 秒调一次 `list`，把这一页没见过的条目与记录排成右下角的浮层提示卡（`shell.overlay` 中 id 为 `superman.toast`，与随包的九个占用者并存）。卡片写明「记忆体已记住」、文件名、分类与大小，点击跳到记忆体面板，7 秒后自动消失，同类卡片最多同时 40 张。**第一次拉取只作为基线**：把当时已有的全部 id 记为已知、不作提示，因此启动时不会被历史文件刷屏，同一个条目只提示一次。面板上传、Chat 上传、Harness 上传与 arXiv 收藏四个来源因此都能被提示到。

6 文献捕获

设置里的 arXiv 检索节（`settings.section`，id 为 `arxiv`）按 arXiv 号、标题、作者、摘要、关键词或分类查找论文，按 arXiv API 的用户手册 [11] 构造查询，并把记录中的会议与期刊信息（`comment`、`journal-ref`）一并显示。取数不由浏览器直连 `export.arxiv.org`：该端点不返回跨域头，因此由本 bundle 的 Host 半在同源路径 `/arxiv/search` 上代理，原样回传 Atom feed，页面再用渲染进程自带的 `DOMParser` 解析。路由只接受「字段 + 检索词 + 页大小 + 排序」四个参数，从不接受 URL，因此不能被当成通用代理；按官方要求，相邻两次上游请求之间默认排队 3000 ms。

每个搜索结果右侧新增一个**收藏到记忆体**按钮，按钮状态为「收藏到记忆体 → 归档中⋯ → 已归档 PDF / 已归档为记录 / 归档失败」。归档逻辑是：

1. 先由页面自己下载论文 PDF。这一步可行的前提是一次实测结论：`arxiv.org` 的 PDF 响应带 `access-control-allow-origin: *`（用 `HEAD` 请求验证），因此页面可以直接取字节，**不需要**为 PDF 再加一条 Host 代理；与之对比，检索用的 `export.arxiv.org` 不带该头，必须走同源代理。
2. 把字节连同元数据（标题、作者、分类、摘要组成的正文）以 `{action:'store', source:'arxiv'}` 存入记忆体；文件名取 arXiv 号（无号则取标题），替换非法字符并截断到 80 字符。于是这篇论文既是一个 PDF 文件，其摘要也能被回忆检索到。
3. 下载被拦或失败时**降级为一条记录**：`{action:'note'}`，id 为 `arxiv:<■>`，标题取论文标题，正文为元数据加摘要，标签取主分类；论文仍然可以被回忆命中，只是没有 PDF 原件。
4. 记忆体本身不可用（插件未装或路由报错）时，按钮报告 ■■■■，不会假装成功。

归档成功后，记忆体的守望者会在下一次轮询时弹出提示卡，论文进入相应分类，之后 Chat 与 Harness 思考时都能回忆到它。第 7 节的实测中，5.0 MB 的 arXiv 论文正是通过这个按钮存入记忆体的。

7 评测

评测分五项：源码规模、离线检查套件、PDF 抽取语料、编码探测、公式电池，另加一组在线部署观察。所有数字都由工作区中的脚本产生或从源码统计得到。

源码规模。 套件共 6 个 JavaScript 文件、7 187 行、294 329 字节，见表 2 与图 2。

表 2. 源码清单：各模块的文件数、行数与字节数。

模块	文件数	行数	字节
chat-mode	2	4,274	178,186
superman	2	1,988	79,879
arxiv-search	2	925	36,264
合计	6	7,187	294,329

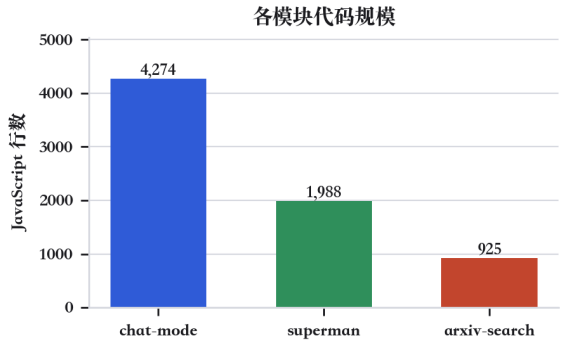


图 2. 各模块的 JavaScript 行数。

其中 `chat-mode` 为 4 274 行（`client.js` 4 017、`index.js` 255），`superman` 为 1 988 行（1 320 + 666），`arxiv-search` 为 925 行（784 + 139）。行数分布直接反映设计约束：浏览器半承担 PDF 解析、公式渲染、Markdown 渲染、编码识别与全部界面，因此远大于 Host 半；三个 Host 半合计仅 1 061 行，只做请求校验、路由转发与存储。

离线检查。 5 套检查共 284 项、1 696 行，全部通过，见表 3 与图 3。全部检查驱动随包发布的**真实文件**。

表 3. 离线检查套件：用途、检查项数与脚本行数。

套件	用途	检查项	行数
chat-mode-check.mjs	管线：路由、SSE、历史、附件、记忆体联动	77	446
chat-mode-format-check.mjs	LaTeX→MathML、Markdown、编码识别	59	264
superman-host-check.mjs	分类、存储、回忆、捕获	39	200
superman-check.mjs	面板：入口、文件夹、预览、上传、提示	57	349
arxiv-smoke.mjs	arXiv 半，含收藏路径	52	451
合计		284	1710

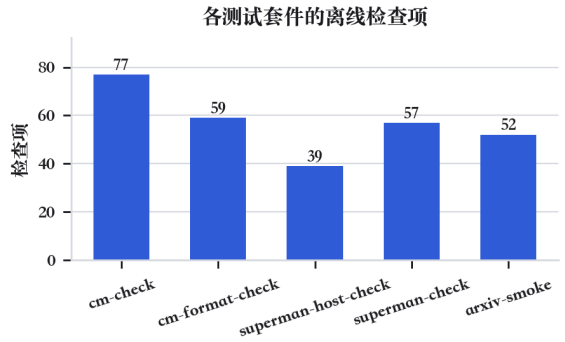


图 3. 各套件的检查项数。

各套件为：`chat-mode-check.mjs` 77 项（管线：路由注册、SSE 分帧、多轮历史、附件内联、图片转附件、PDF 抽取、笔记本拍平、会话记录的读写与配额裁剪、失败路径与面板渲染）；`chat-mode-format-check.mjs` 59 项（LaTeX→MathML 的逐项断言与公式清单、Markdown 块结构、`$100` 不误判、编码识别、GBK 笔记本端到端、网页去标签）；`superman-host-check.mjs` 39 项（分类、存储、图片尺寸、回忆、捕获、错误路径）；`superman-check.mjs` 57 项（入口、图标、字典完整性、文件夹、预览、上传、删除、提示）；`arxiv-smoke.mjs` 52 项（Host 路由的校验、转发、限速、查询构造、页大小钳制、拒绝码与上游失败，以及客户端的三条归档路径）。

验证方法学界定了这些数字的含义。Host 半在一个假的 Cordis 上下文里运行，用临时记忆目录；浏览器半用一组替身装载——桩模块加载器 `window.__ModuleLoader__`、桩 React（函数组件被直接调用，因此可在 Node 里断言返回的节点树）、内存版 `localStorage` 与被替换的 `fetch`。这些检查因此证明：两半在通信形状上一致、解析与渲染逻辑在给定输入上产生预期结果、会话记录能在重载后复原、配额裁剪按预期丢弃最旧会话；但它们**不能**证明浏览器里的真实 DOM 结构、样式与交互正确，也不能替代端到端浏览器测试（第 8 节列为效度威胁）。

PDF 抽取语料。 语料为 14 个真实文件，含 5 份中文文档与 9 份 Apple 系统生成的矢量图形 PDF，另有一篇 arXiv 预印本 [12] 作为记忆体的收录样本（见下文在线观察）。总体结果：199 942 个字符、67 235 个汉字、138 页，替换字符与 C1 控制字符合计 0；5 份文档抽到文本层，9 份被如实报告为 `no-text`。逐文件结果见表 4，字符量与页数的关系见图 4，耗时与页数的关系见图 5。

表 4. 真实语料上的 PDF 抽取结果。

文件（化名）	KB	毫秒	字符	汉字	页	备注
brochure.pdf	1507	38.9	9,423	9	19	
document-B.pdf	177	7.9	786	394	2	
prospectus-2025.pdf	576	76.4	90,938	31,791	56	
prospectus-2026.pdf	567	75.1	89,764	31,252	54	
contract-2027.pdf	203	29.0	36,611	0	8	
文本层缺失	—	—	0	0	0	9 files

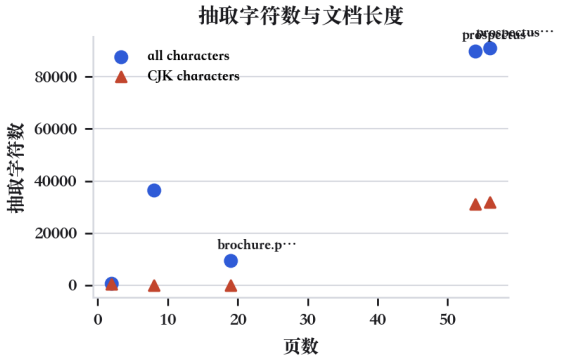


图 4. 各语料文件抽出的字符数与页数。

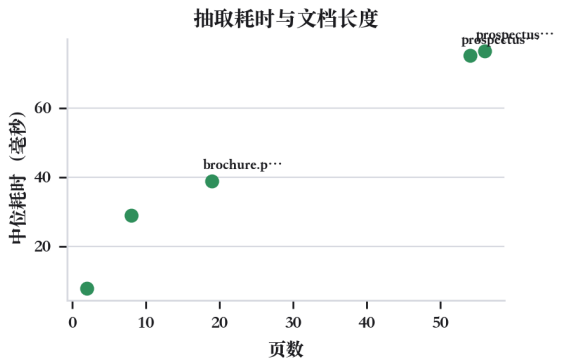


图 5. 中位抽取时间与文档页数。

延迟方面：19 页、中文占比极低的 `brochure.pdf` 中位耗时 34.7 ms，抽出 9 423 个字符（汉字仅 9 个，说明其主体文字层并非中文）；两份 54 – 56 页的中文招生简章分别用 71.8 ms 与 71.3 ms，抽出 90 938 与 89 764 个字符、31 791 与 31 252 个汉字，约每页 1 600 余字符；7 页合同 11.7 ms；2 页成绩单 7.3 ms；9 份矢量图形文件各自 0.2 – 0.8 ms，字符数为 0，报告 `no-text`。

「0 个乱码字符」这一项值得说明：抽取失败的文档应当报告失败，而不是输出替换字符。语料中 9 份图形文件的页树里根本没有文本算符，实现因此走了 `no-text` 分支；若设计成「尽量输出」，这 9 份文件会产生数万个替换字符或 C1 控制字符。噪声列全为 0 说明失败被正确地报告为失败。

编码探测。 7 个探针全部识别正确，见表 5 之前的表 6 与图 6。

表 6. 编码探测：7 个探针的预期、实际与可信度。

探测样本	期望	识别结果	正确
UTF-8	utf-8	utf-8	☐
UTF-8 BOM	utf-8	utf-8	☐
GBK	gbk	gbk	☐
Big5	big5	big5	☐
UTF-16LE (no BOM)	utf-16le	utf-16le	☐
PNG header	binary	binary	☐
random high bytes	binary	binary	☐

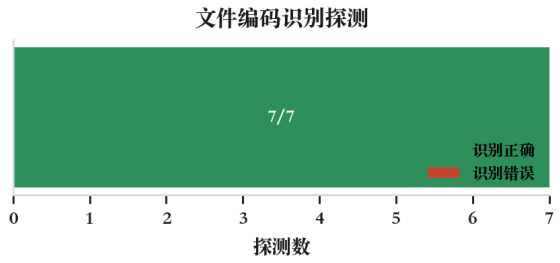


图 6. 编码探测的正确项数。

探针覆盖 UTF-8、UTF-8 BOM、GBK（a 加 `C4 E3 BA C3`）、Big5（`A4 A4 A4 E5`）、无 BOM 的 UTF-16LE、PNG 文件头与随机高字节串。前三者与无标记 UTF-16 的可信度分别为 1.769、1.0、2.333、3.0 与 1.8；PNG 头与随机高字节串的可信度为 0，被正确判为二进制而未被内联。GBK 与 Big5 两个探针的可信度均高于 UTF-8 探针，说明「严格 UTF-8 解码失败后按候选集打分」这条路径是靠分数而不是靠枚举顺序作出判断的。

公式电池。 16 条表达式全部生成结构合法的 `<math>` 标记，且全部没有 LaTeX 反斜杠残留，见表 5。电池覆盖分式、根式、上下标、大算符（求和、积分、环路积分）、矩阵与 `aligned`/`cases` 环境、核素记法 `{}^{238}_{92}\mathrm{U}`、向量与重音、`\mathbb`/`\mathcal`、`\left…\right` 伸缩括号与带括号的单位矩阵。标记长度从 148 到 436 字符；`\frac` 类含 1 – 2 个分式节点，`\sum`、`\int`、`\oint`、`\lim` 类含 3 – 4 个上下标节点，三个环境表达式各含 1 个表格节点。

在线部署观察。 在运行中的页面上观察到：三个侧边栏条目 `conversation`（−20）、`chat`（−10）、`superman`（−5）与随包的 `plugins`（0）并列生效；提示卡浮层以 `shell.overlay[superman.toast]` 注册，与随包的九个占用者并存；记忆体的动态提示词分段出现**在这套扩展自身所在的 Agent 会话的系统提示词里**，列出了实时的记忆目录路径与计数（`PDF 2 · ■■ 0 · ■■ 0 · ■■ 0 · ■■/■■■■ 2`）——Harness 模式的回忆路径因此由本会话本身所验证。开发中共有两个 PDF 进入记忆体：一份 203.4 KB 的扫描件（经面板上传，因无文字层只保留文件）与一篇 5.0 MB 的 arXiv 论文 [12]（经收藏按钮存入，同时带有元数据与摘要正文）。

8 讨论、效度威胁与局限

为什么是槽位而不是替换。 套件没有替换任何随包 UI，所有条目都是新增，好处是随宿主升级保持可用，且从 profile 列表移除即卸载。代价是无法调整宿主既有面板的内部布局；`conversation` 条目不渲染内容，正是因为「选中宿主自己的会话面板」只能通过占用槽位条目表达。

为什么是已鉴权路由而不是新端口。 走 `/api` 通道意味着继承凭据与来源校验、不引入端口冲突、随插件卸载自动撤销。代价是形态上的限制：只能是精确路径，不能自定义中间件或长连接协商；SSE 的读端要由页面自己实现，因此帧的分隔符约定（`\n\n`）成为契约的一部分，两半必须同时遵守。相对地，自行监听端口需要在插件里处理凭据、跨域与生命周期，并把「同源」这一最强的前提丢掉。

浏览器半不能 import 宿主 Client 包，代价是什么。 浏览器半是普通 ES 模块，模块图只提供它自己与 React，因此宿主内部已有的能力都不能复用。代价有三处：必须自己写 PDF 阅读器（本文技术细节最密集的部分，第 4 节的五类缺陷全部来自它）、自己写公式渲染器而不是复用宿主 Markdown 组件、自己写编码探测器而不是依赖 `File.text()`。收益是 bundle 自包含：记忆体与 Chat 都不依赖宿主内部模块的私有导出，不会因宿主的内部重构而失效。

效度威胁。 其一，验证是离线套件加在线槽位与提示词观察，**没有浏览器级（端到端 DOM）自动化测试**：桩 React 直接调用函数组件，能断言节点树与属性，但不能断言真实布局、滚动、拖拽与粘贴、字体回退的实际命中等视觉与交互细节，因此这些细节**未经核实**。其二，PDF 语料只有 14 个文件、全部来自本机，中文文档集中于两份招生简章与一份合同，不能代表 PDF 生态的多样性。其三，编码探测只有 7 个探针，只验证「识别为正确标签」，不验证长文本上的稳定性。其四，公式电池的 16 条表达式由作者挑选，且断言的是「标记合法、无反斜杠泄漏」而非排版保真度，不能说明与 TeX 的视觉一致性。其五，在线观察是单次部署的快照，样本量为 1。

局限。 记忆体与 Chat 的若干能力有明确边界：

• PDF 抽取是**文字层读取器**：扫描件或轮廓化（转曲）文档抽不到文本，被报告为无文本（语料 14 份中有 9 份）。

• 回忆是**词法**的（词元重叠加时间新鲜度），不是基于嵌入向量的；它确定、便宜，但只能捕捉表层相似度。中文用单字与二元组，不常见词组的召回取决于二元组是否恰好重合。

• Chat 历史存放在页面 origin 的 `localStorage`：能跨重启存活，但不能跨清除站点数据，也不与另一个端口上的 `dsh web` 共享。

• 从历史恢复的图片只保留 160 px 缩略图；重发旧消息时原图被替换成一句说明，而不是被静默丢弃。

• 验证手段是离线套件加在线检查，没有浏览器级自动化测试，因此真实布局与交互细节未经核实。

• 记忆体除上限（条目数与记录数有界）外没有压缩策略，去重仅靠内容哈希加文件名：内容相同但文件名不同的材料会被存两次，索引会持续增长。

9 结论与未来工作

本文报告了一套为智能体编码框架构建的扩展套件，它用一个纯对话面板、一份跨模式共享的记忆体与一条文献捕获路径，补上了宿主界面在对话、记忆与资料汇集三方面的缺口，并且不修改宿主源码、不替换任何随包 UI、不新开监听端口。实现上的核心取舍是：把界面放进槽位，把通信放进已鉴权的同源路由，把浏览器半做成自包含的模块——因此 PDF 文本抽取、LaTeX→MathML 渲染与编码识别都必须自己实现，其正确性由 284 项离线检查与 14 份真实 PDF、7 个编码探针、16 条公式所约束。

未来工作有四个方向。其一，**基于嵌入向量的回忆**：用本地嵌入模型或宿主提供的向量服务替换词法打分，同时保留词法打分作为无模型时的回退，以覆盖「说法不同但意思相同」的召回。其二，**OCR 回退**：对报告 `no-text` 的文件，在用户明确要求时调用 OCR，把结果作为低可信度的正文预览存入记忆体并标明来源。其三，**跨 origin 的记忆同步**：记忆体已跨 origin 共享（它存在 Host 侧目录里），但 Chat 历史不是；把会话记录持续镜像进记忆体，可让用户在任一端口看到一致的对话历史。其四，**打包为 profile bundle 集合**：三个 bundle 目前分别安装，可合并成一个可一键安装、带版本约束与依赖声明的集合，作为插件市场上的一种分发形态。

致谢

感谢 DeepSeek Harness 与 Cordis 的作者提供可扩展的宿主与插件框架；感谢 STIX 项目提供本文排版所依赖的数学字体。本文的实测数字均来自本工作区中的脚本与源码统计，未使用任何外部基准。

参考文献

1. DeepSeek Harness — https://github.com/deepseek-ai/deepseek-harness
2. Cordis plugin framework — https://github.com/cordiverse/cordis
3. ISO 32000-1:2008, Document management — Portable document format — Part 1: PDF 1.7.
4. Adobe Systems, PDF Reference, 6th edition (FlateDecode, ToUnicode CMaps) — https://opensource.adobe.com/dc-acrobat-sdk-docs/pdfstandards/pdfreference1.6.pdf
5. W3C, MathML Core — https://www.w3.org/TR/mathml-core/
6. WHATWG, Encoding Standard (GBK, Big5, Shift_JIS, EUC-KR decoders) — https://encoding.spec.whatwg.org/
7. WHATWG, HTML Living Standard, § Server-sent events — https://html.spec.whatwg.org/multipage/server-sent-events.html
8. D. E. Knuth, The TeXbook, Addison-Wesley, 1984.
9. KaTeX — https://katex.org/
10. S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond", Foundations and Trends in Information Retrieval, 3(4):333 – 389, 2009.
11. arXiv API user manual — https://info.arxiv.org/help/api/user-manual.html
12. OpenAI et al., "GPT-4 Technical Report", arXiv:2303.08774, 2023. (used only as a corpus sample, not as a design claim)
13. STIX Two Math font — https://github.com/stipub/stixfonts
14. React — https://react.dev/